

Python

Drugi wykład

dr inż. Marcin Ciura, Uniwersytet Komisji Edukacji Narodowej

Przepraszam was za to,
że będę do was mówić przez „wy” :-)

Niektóre objaśnienia i przykłady pochodzą
z kursu dr. hab. Grzegorza Góralskiego
Jego kurs jest dostępny pod adresem
<https://ggoralski.gitlab.io/python-wprowadzenie/>

Proszę robić notatki :-)

Plan na dziś

Dzisiaj przedstawię wam:

- + Funkcję **print**
- + Funkcje
- + Łańcuchy znaków, czyli teksty
- + Instrukcję przypisania
- + Słowa kluczowe
- + Funkcję **input**

- + Indeksowanie łańcuchów
- + Wycinki łańcuchów
- + Dodawanie łańcuchów
- + Metody łańcuchów
- + Instrukcję warunkową **if**
- + Operatory porównania
- + Operatory **in** i **not in**
- + Operatory logiczne
- + I pliki tekstowe

Zadanie na rozgrzewkę:
Proszę narysować plik
Każda odpowiedź jest dobra :-)

Wróćimy do plików później

Funkcja print

Umówmy się

Umówmy się, że na slajdach prezentacji:

- + fragmenty programów w Pythonie będą zielone
- + wyniki, które te programy wypisują na ekranie będą różowe
- + dane, które wprowadza użytkownik programu, będą niebieskie

Co to jest funkcja w językach programowania?

Funkcja to osobny fragment programu, który działa na określonych danych

Każdą funkcję możemy wywołać, czyli spowodować jej wykonanie. Za każdym razem, gdy wywołujemy funkcję, możemy przekazać jej inne dane

Funkcja print

Funkcja **print** wypisuje tekst

Tak wywołujemy (my i program) funkcję **print**:

```
print('Witaj, piękny świecie :-')
```

Witaj, piękny świecie :-)

Funkcja print

```
print('Witaj, piękny świecie :-')
```

Witaj, piękny świecie :-)

Aby wywołać funkcję **print**, napisaliśmy:

+ nazwę funkcji

+ nawiasy (), a wewnątrz nich argument funkcji

Wywołaliśmy funkcję **print** z jednym argumentem. Tym argumentem jest tekst Witaj, piękny świecie :-)

Funkcje

Z czego składa się funkcja?

- + Każda funkcja ma nazwę
- + Każda funkcja może mieć dane wejściowe. Mówimy na te dane wejściowe argumenty funkcji
- + Każda funkcja ma kod, czyli instrukcje, które są wykonywane przez program, kiedy wywołujemy tę funkcję

Wywoływanie funkcji

Aby program wykonał funkcję, powinniśmy (my i program) wywołać tę funkcję

```
print('2 + 2 =', 2+2)
```

2 + 2 = 4

Wywoływanie funkcji

```
print('2 + 2 =', 2+2)
```

2 + 2 = 4

Aby wywołać funkcję, piszemy kolejno:

- + nazwę funkcji

- + nawiasy ()

- + a wewnątrz tych nawiasów argumenty funkcji,
rozdzielone przecinkami ,

Definiowanie funkcji

Każda funkcja jest gdzieś zdefiniowana, czyli gdzieś istnieje jej definicja

```
def wypisz_sumę(a, b):  
    print(a, '+', b, '=', a+b)
```

Definiowanie funkcji

```
def wypisz_sumę(a, b):  
    print(a, '+', b, '=', a+b)
```

Definicja funkcji określa:

- + nazwę funkcji
- + argumenty funkcji
- + kod funkcji

Definiowanie funkcji

Kiedy Python widzi definicję funkcji, nie wykonuje tej funkcji, tylko zapamiętuje jej nazwę, argumenty i kod

Dopiero potem możemy (my i program) wywołać tę funkcję, czyli podać jej nazwę i argumenty, aby wykonać jej kod

Definiowanie funkcji

Kiedy wywołujemy funkcję, powinniśmy podać tyle samo argumentów, co wtedy, gdy ją definiowaliśmy

Kiedy program wywołuje funkcję, przypisuje wartości jej argumentów w miejscu wywołania do argumentów podanych w jej definicji

Definiowanie i wywoływanie funkcji — przykład

```
def wypisz_sumę(a, b):  
    print(a, '+', b, '=', a+b)
```

```
wypisz_sumę(2, 2)
```

```
wypisz_sumę(3, 6)
```

2 + 2 = 4

3 + 6 = 9

Funkcje wbudowane, funkcje biblioteczne i funkcje definiowane przez użytkownika

Funkcje można podzielić na trzy grupy:

1. Definicje najpotrzebniejszych funkcji są wbudowane w Pythona. Taka jest na przykład funkcja `print`

Funkcje wbudowane, funkcje biblioteczne i funkcje definiowane przez użytkownika

2. Definicje wielu innych funkcji są umieszczone w bibliotekach funkcji.

Slogan Pythona „baterie są w zestawie” oznacza, że biblioteki funkcji dostarczane wraz z Pythonem przydają się w wielu zastosowaniach

Funkcje wbudowane, funkcje biblioteczne i funkcje definiowane przez użytkownika

3. Definicje pozostałych funkcji są tworzone przez użytkowników Pythona

Funkcje zwracające wynik i funkcje z efektami ubocznymi

Są dwa rodzaje funkcji:

- + Takie funkcje, które zwracają jakiś wynik. Takie są na przykład funkcje matematyczne z biblioteki `math`: `math.sin`, `math.cos` i inne
- + Takie funkcje, które nie zwracają wyniku, ale za to zmieniają coś dookoła siebie, na przykład na ekranie. Taka jest na przykład funkcja `print`

łańcuchy znaków, czyli teksty

Łańcuchy znaków, czyli teksty

Aby komputer mógł odróżnić teksty, które wypisuje na ekranie, od innych części programu, umieszczamy każdy tekst wewnątrz cudzysłówów:

- + pojedynczych 'to jest przykład tekstu'
- + albo podwójnych "to też jest przykład tekstu"

Cudzysłowy wokół tekstu nie wchodzą w skład tekstu, tylko oznaczają jego początek i koniec

Łańcuchy znaków, czyli teksty

Inna nazwa tekstu to łańcuch znaków, w skrócie łańcuch

(Po angielsku łańcuch znaków nazywa się „string”, czyli „sznurek”)

Znakiem może być litera, cyfra, znak przestankowy

Są nawet białe znaki: spacja, znak tabulacji, znak nowej linii. Powiem o nich potem

Łańcuchy znaków, czyli teksty

Cudzysłowy ' lub " oznaczają początek i koniec łańcucha.
Oba rodzaje cudzysłowów są dobre

Wewnątrz łańcucha możemy umieszczać cudzysłowy, ale
cudzysłowy wewnątrz łańcucha muszą być inne niż te
cudzysłowy, które oznaczają początek i koniec tego
łańcucha

Łańcuchy znaków, czyli teksty

Wewnątrz łańcucha możemy umieszczać cudzysłowy, ale cudzysłowy wewnątrz łańcucha muszą być inne niż te cudzysłowy, które oznaczają początek i koniec tego łańcucha

```
print('Autorem książki "Trzej muszkietierowie" jest Alexandre Dumas.')
```

Autorem książki "Trzej muszkietierowie" jest Alexandre Dumas.

```
print("D'Artagnan był muszkietierem.")
```

D'Artagnan był muszkietierem.

Łańcuchy znaków, czyli teksty

A jednak możemy umieścić w łańcuchu taki sam cudzysłów jak te cudzysłowy, które otaczają ten łańcuch, pod warunkiem, że poprzedzimy ten cudzysłów znakiem lewego ukośnika \

```
print("\Monty Python i Święty Graal\" to komedia filmowa z  
1975 roku.")
```

"Monty Python i Święty Graal" to komedia filmowa z 1975 roku.

(Po angielsku lewy ukośnik nazywa się „backslash”)

Łańcuchy znaków, czyli teksty

Dzięki lewemu ukośnikowi \ możemy wstawiać do łańcuchów takie znaki, które trudno zapisać bezpośrednio, na przykład:

- + \' w zapisie łańcucha oznacza pojedynczy cudzysłów ' w łańcuchu
- + \" w zapisie łańcucha oznacza podwójny cudzysłów " w łańcuchu
- + \\ w zapisie łańcucha oznacza lewy ukośnik \ w łańcuchu
- + \n w zapisie łańcucha oznacza znak nowej linii w łańcuchu
- + \t w zapisie łańcucha oznacza znak tabulacji w łańcuchu

Łańcuchy znaków, czyli teksty

- + `\'` w zapisie łańcucha oznacza pojedynczy cudzysłów `'` w łańcuchu
- + `\"` w zapisie łańcucha oznacza podwójny cudzysłów `"` w łańcuchu
- + `\\` w zapisie łańcucha oznacza lewy ukośnik `\` w łańcuchu
- + `\n` w zapisie łańcucha oznacza znak nowej linii w łańcuchu
- + `\t` w zapisie łańcucha oznacza znak tabulacji w łańcuchu

```
print('raz\n dwa\n trzy')
```

```
raz  
dwa  
trzy
```

Instrukcja przypisania

Instrukcja przypisania

```
tekst = 'Ahoj, krásný světe :-)'  
print(tekst)
```

Ahoj, krásný světe :-)

Instrukcja przypisania **zmienna = obiekt** wiąże zmienna, czyli nazwę, z obiektem, czyli z wartością. Kiedy potem używamy tej nazwy, jest tak, jakbyśmy używali tego obiektu

Instrukcja przypisania

Program może zmieniać wartość zmiennej

W każdej chwili zmienna ma tylko jedną wartość

```
tekst = 'Ahoj, krásný světe :-)
```

```
print(tekst)
```

```
tekst = 'Witaj, piękny świecie :-)
```

```
print(tekst)
```

Ahoj, krásný světe :-)

Witaj, piękny świecie :-)

Nazwy zmiennych i funkcji

Nazwy

Nazwa zmiennej lub funkcji może się składać z:

- + liter, w tym polskich liter
- + cyfr od 0 do 9
- + znaków podkreślenia _

Nazwa zmiennej lub funkcji nie może zawierać spacji

Nazwa zmiennej lub funkcji nie może się zaczynać od cyfry, czyli może się zaczynać od litery albo od znaku podkreślenia

Nazwy

W nazwach odróżniamy duże litery od małych liter

Na przykład nazwisko, Nazwisko i NAZWISKO to trzy różne nazwy

Dobre zwyczaje

Dobrzy programiści mają dobre zwyczaje:

- + zaczynamy nazwę każdej zmiennej i funkcji małą literą
- + łączymy nazwy złożone z wielu wyrazów znakami podkreślenia _
- + nazywamy zmienne rzeczownikami, na przykład `gatunek, liczba_prób`
- + nazywamy funkcje czasownikami w trybie rozkazującym, na przykład `wczytaj_gatunek, oblicz_podatek`

Słowa kluczowe

Słowa kluczowe

Znaczenie każdej nazwy w programie ustalają programiści

Słowa kluczowe wyglądają podobnie do nazw, ale znaczenie słów kluczowych ustalili twórcy Pythona. Nie da się zmienić tego znaczenia

W Pythonie jest trzydzieści kilka słów kluczowych, na przykład if, for, def

W Pythonie nie możemy używać słów kluczowych jako nazw

Umówmy się, że na slajdach słowa kluczowe będą podkreślone

Komentarze

Każdy komentarz to część programu przeznaczona dla czytelników programu. Komputer nie czyta komentarzy

Umówmy się, że komentarze na slajdach będą niebieskie

Każdy komentarz zapisujemy od znaku kratki do końca linii

Łańcuchy, nazwy, słowa kluczowe
Podsumowanie

Łańcuchy, nazwy, słowa kluczowe — podsumowanie

'To jest łańcuch znaków, czyli tekst'

"To też jest łańcuch znaków"

'To są znaki specjalne: \nznak nowej linii, \\ lewy ukośnik'

to_jest_nazwa

def if for # To są słowa kluczowe

Funkcja print — uzupełnienie

Funkcja `print` — uzupełnienie

Funkcja `print` może mieć 0 argumentów, 1 argument albo więcej niż 1 argument. Kiedy wypisuje te argumenty, oddziela je spacjami. Kiedy wypisze wszystkie argumenty, kończy pisać w tej linii. Programiści mówią: „przechodzi do nowej linii”

Kiedy następny raz wywołamy funkcję `print`, ta funkcja zacznie pisać od następnej linii

Funkcja print — uzupełnienie

```
print('Cześć :-)')  
print() # Wypisz pustą linię  
print('Jak się masz?')  
Cześć :-)
```

Jak się masz?

Funkcja print — uzupełnienie

```
# Wypisz dwa argumenty, oddzielając je od siebie  
# pojedynczymi znakami spacji,  
# a potem przejdź do nowej linii  
miasto = 'Kraków'  
print(miasto, 'to ładne miasto')  
Kraków to ładne miasto
```

Funkcja input

Funkcja `input`

Dzięki funkcji `input` program wczytuje teksty z klawiatury

Funkcja `input`:

- + wyświetla swój argument na ekranie
- + czeka, aż użytkownik wpisze tekst i naciśnie klawisz `Enter`
- + zwraca wpisany tekst

Funkcja input

Dzięki funkcji `input` program wczytuje teksty z klawiatury

```
imię = input('Jak masz na imię? ')
```

```
print(imię, 'to ładne imię :-')
```

Jak masz na imię? Eleonora

Eleonora to ładne imię :-)

Postęp wykładu

Przedstawiłem wam:

- + Funkcję **print**
- + Funkcje
- + Łańcuchy znaków, czyli teksty
- + Instrukcję przypisania
- + Słowa kluczowe
- + I funkcję **input**

Teraz omówię:

- + Indeksowanie łańcuchów
- + Wycinki łańcuchów
- + Dodawanie łańcuchów
- + I metody łańcuchów

Proszę mi zadać pytanie :-)

Jaki polski wyraz brzmi
najładniej w Twoich uszach?

Według pewnego Anglika najładniejszy polski wyraz to nie miłość

Według pewnego Anglika najładniejszy polski wyraz to nie miłość

To

cieleęcina :-)

Indeksowanie łańcuchów

Indeksowanie łańcuchów

Każdy znak łańcucha ma swój numer, czyli indeks

Odczytujemy pojedynczy znak łańcucha, podając:

+ nazwę tego łańcucha albo po prostu ten łańcuch
+ indeks tego znaku w nawiasach kwadratowych []

0	1	2	3	4	5	6	7	8
c	i	e	l	ę	c	i	n	a
-9	-8	-7	-6	-5	-4	-3	-2	-1

Indeksowanie łańcuchów

Pierwszy znak ma indeks 0

Drugi znak ma indeks 1

Trzeci znak ma indeks 2

i tak dalej...

```
tekst = 'cieęcina'
```

```
print(tekst[2])
```

e

Programiści tak czytają zapis

łańcuch[2]:

„łańcuch pod indeksem dwa”

0	1	2	3	4	5	6	7	8
c	i	e	ł	ę	c	i	n	a
-9	-8	-7	-6	-5	-4	-3	-2	-1

Indeksowanie łańcuchów

Dla wygody możemy liczyć indeksy od końca łańcucha, używając liczb ujemnych

Ostatni znak ma indeks -1.
Przedostatni znak ma indeks -2
i tak dalej...

```
tekst = 'ciełęcina'  
print(tekst[-2], tekst[-5])
```

ne

0	1	2	3	4	5	6	7	8
c	i	e	ł	ę	c	i	n	a
-9	-8	-7	-6	-5	-4	-3	-2	-1

Indeksowanie łańcuchów

```
tekst = 'ciełęcina'
```

Próba użycia indeksu
większego lub równego
długości łańcucha
powoduje wyjątek

```
print(tekst[100])  
IndexError
```

0	1	2	3	4	5	6	7	8
c	i	e	ł	ę	c	i	n	a
-9	-8	-7	-6	-5	-4	-3	-2	-1

Indeksowanie łańcuchów

```
tekst = 'ciełęcina'
```

Próba użycia indeksu
mniejszego niż minus
długość łańcucha też
powoduje wyjątek

```
print(tekst[-17])  
IndexError
```

0	1	2	3	4	5	6	7	8
c	i	e	ł	ę	c	i	n	a
-9	-8	-7	-6	-5	-4	-3	-2	-1

Jak wygląda wyjątek?

```
print(tekst[-17])
```

Jeśli coś spowodowało wyjątek, to Python wypisze na ekranie raport o tym wyjątku

Na dole raportu znajdziemy opis tego wyjątku

`IndexError: string index out of range`

Wycinki tańcuchów

Wycinki łańcuchów

Odczytujemy dowolnie wiele sąsiednich znaków łańcucha, czyli wycinek łańcucha, podając w nawiasach kwadratowych [] indeksy początku wycinka i końca wycinka, rozdzielone dwukropkiem

```
tekst = 'ciełęcina'  
print(tekst[0:5])  
ciełę
```

0	1	2	3	4	5	6	7	8	9
ciełęcina									
-9	-8	-7	-6	-5	-4	-3	-2	-1	

Wycinki łańcuchów

Najwygodniej zapamiętać, że indeks początku wycinka i indeks końca wycinka to indeksy miejsc między znakami łańcucha

```
tekst = 'ciełęcina'  
print(tekst[5:7], tekst[-4:-2])  
ci ci
```

0	1	2	3	4	5	6	7	8	9
ciełęcina									
-9	-8	-7	-6	-5	-4	-3	-2	-1	

Wycinki łańcuchów

Gdy w wycinku podamy taki indeks początku albo taki indeks końca, któremu odpowiada znak poza łańcuchem, to wyjątek nie wyskoczy

```
tekst = 'ciełęcina'  
print(tekst[-300:5], tekst[-300:300])  
ciełę cieleęcina
```

0	1	2	3	4	5	6	7	8	9
ciełęcina									
-9	-8	-7	-6	-5	-4	-3	-2	-1	

Wycinki łańcuchów

Nie ma wyjątku, gdy
w wycinku podamy indeks
początku większy od indeksu
końca

```
tekst = 'ciełęcina'  
print(tekst[5:2])
```

Wtedy wycinek jest pusty

0	1	2	3	4	5	6	7	8	9
ciełęcina									
-9	-8	-7	-6	-5	-4	-3	-2	-1	

Wycinki łańcuchów

W wycinkach możemy pomijać indeks początku. Wtedy Python przyjmuje, że indeks początku to zero

```
tekst = 'ciełęcina'  
print(tekst[:5], tekst[:-4])  
ciełę ciele
```

0	1	2	3	4	5	6	7	8	9
ciełęcina									
-9	-8	-7	-6	-5	-4	-3	-2	-1	

Wycinki łańcuchów

W wycinkach możemy pomijać indeks końca. Wtedy Python przyjmuje, że indeks końca to długość łańcucha

```
tekst = 'cielęcina'  
print(tekst[7:], tekst[-2:])  
print(tekst[:])
```

```
na na  
cielęcina
```

0	1	2	3	4	5	6	7	8	9
cielęcina									
-9	-8	-7	-6	-5	-4	-3	-2	-1	

Dodawanie tańców

Dodawanie łańcuchów

Łańcuchy możemy dodawać do łańcuchów:

```
rebus = 'cielęcina'
```

```
print('m' + rebus[1:5] + ' szp' + rebus[-3:] + 'k')
```

miele szpinak

Metody łańcuchów

Metody

Metoda to taka funkcja, która jest powiązana z obiektem

Metody obiektu wywołuje się, pisząc kolejno:

- + nazwę tego obiektu albo po prostu ten obiekt
- + kropkę .
- + nazwę metody
- + nawiasy (), a wewnątrz nich argumenty metody, rozdzielone przecinkami ,

Umówmy się, że na slajdach nazwy metod będą miały na początku kropkę

Łańcuchy — metoda `.upper`

Wszystkie metody łańcuchów zwracają jakieś wyniki

Na przykład metoda `.upper` zwraca nowy łańcuch. W tym łańcuchu wszystkie małe litery są zamienione na wielkie litery

łańcuchy — metoda .upper

Metoda .upper zwraca nowy łańcuch. W tym łańcuchu wszystkie małe litery są zamienione na wielkie litery

```
tekst = 'ciełęcina'  
nowy_tekst = tekst.upper()  
print(nowy_tekst)  
CIEŁĘCINA
```

Łańcuchy — metoda .lower

Metoda `.lower` zwraca nowy łańcuch. W tym łańcuchu wszystkie wielkie litery są zamienione na małe litery

```
tekst = 'Dziennik Polski'  
tekst = tekst.lower()  
print(tekst)  
dziennik polski
```


Łańcuchy — metoda `.strip`

Metoda `.strip` zwraca nowy łańcuch bez początkowych i końcowych białych znaków

Białe znaki to:

- + spacje
- + znaki tabulacji `\t`
- + znaki nowej linii `\n`

Łańcuchy — metoda `.strip`

Metoda `.strip` zwraca nowy łańcuch bez początkowych i końcowych białych znaków

```
tekst = '\tAnna Kowalska \n'  
print('[' + tekst.strip() + ']')  
[Anna Kowalska]
```


Typ logiczny bool

Obiekty, które dotychczas widzieliśmy, to łańcuchy znaków

Oprócz nich są obiekty innych typów, między innymi typu logicznego, czyli typu bool

Są 2 obiekty typu logicznego: **True**, czyli prawda, i **False**, czyli nieprawda

(Typ logiczny nazywa się po angielsku „Boolean type”, czyli typ Boolowski, od nazwiska angielskiego matematyka George’a Boole’a)

Łańcuchy — metoda .startswith

Metoda .startswith sprawdza, czy dany łańcuch znaków zaczyna się od innego łańcucha znaków. Zwraca True albo False

```
tekst = 'ciełęcina'  
print(tekst.startswith('ciel'))  
print(tekst.startswith('ć'))
```

True

False

Łańcuchy — metoda .endswith

Metoda .endswith sprawdza, czy dany łańcuch znaków kończy się innym łańcuchem znaków. Zwraca True albo False

```
tekst = 'ciełęcina'  
print(tekst.endswith('ć'))  
print(tekst.endswith('na'))
```

False

True

Przypisanie wielu wartości do wielu zmiennych

Możemy naraz przypisywać wiele wartości do wielu zmiennych pod warunkiem, że wartości jest tyle samo, co zmiennych

`imię, drugie_imię, nazwisko = 'Anna', 'Maria', 'Kowalska'`

Próba takiego przypisania, w którym wartości nie jest tyle samo, co zmiennych, powoduje wyjątek

Łańcuchy — metoda `.split`

Metoda `.split` dzieli łańcuch na części złożone z takich znaków, które nie są białymi znakami

```
dane_osobowe = 'Anna Maria Kowalska\n'  
imię, drugie_imię, nazwisko = dane_osobowe.split()
```

Łańcuchy — metoda .split

```
dane_osobowe = ' Anna Maria Kowalska\n'  
imię, drugie_imię, nazwisko = dane_osobowe.split()  
print(imię)  
print(drugie_imię)  
print(nazwisko)
```

Anna

Maria

Kowalska

Funkcja wbudowana len

Funkcja **len** zwraca długość łańcucha

```
tekst = 'pies'
```

```
print(len(tekst))
```

```
tekst = " # To są dwa pojedyncze cudzysłowy
```

```
print(len(tekst))
```

4

0

Indeksowanie, wycinki, dodawanie,
metody łańcuchów
Podsumowanie

Indeksowanie, wycinki, dodawanie, metody łańcuchów — podsumowanie

```
tekst = 'łańcuch'  
print(tekst[0])  
print(tekst[1:3])  
print(tekst[:4])  
print(tekst[-3:])  
print(tekst[-3:] + 'o')
```

```
print('Python'.upper())  
print('Python'.lower())  
print('Python\n'.strip())  
print('Python'.startswith('Py'))  
print('Python'.endswith('hon'))  
imię, nazwisko = 'Jan Kot\n'.split()  
print(len('Python'))
```

Postęp wykładu

Przedstawiłem wam:

- + Funkcję **print**
- + Funkcje
- + Łańcuchy znaków, czyli teksty
- + Instrukcję przypisania
- + Słowa kluczowe
- + Funkcję **input**
- + Indeksowanie łańcuchów

- + Wycinki łańcuchów
- + Dodawanie łańcuchów
- + I metody łańcuchów

Teraz omówię:

- + Instrukcję warunkową **if**
- + Operatory porównania
- + Operatory **in** i **not in**
- + I operatory logiczne

Proszę mi zadać pytanie :-)

Instrukcja warunkowa if

Instrukcja warunkowa if

Instrukcja warunkowa to:

- + Słowo kluczowe if
- + Wyrażenie logiczne. Jego wynikiem powinno być **True** albo **False**
- + Dwukropek :
- + Nowa linia
- + Kilka spacji albo znak tabulacji, tak zwane wcięcia
Wcięcia są ważne :-)
- + Instrukcje przesunięte w prawo. Te instrukcje wykonają się tylko wtedy, gdy wartością wyrażenia logicznego jest **True**

Instrukcja warunkowa if

```
test = True
```

```
if test:
```

```
    print('Prawda')
```

Prawda

Instrukcja warunkowa if

```
test = True
```

```
if test:
```

```
    print('Prawda')
```

Prawda

```
test = False
```

```
if test:
```

```
    print('Prawda')
```

Operator

Operatory

Operatory to takie symbole lub słowa kluczowe, dzięki którym program wykonuje działania na obiektach

Operatory porównania

Operatory porównania

Dzięki operatorowi `==` program porównuje dwie wartości.
Wynik operatora `==` to:

- + `True`, gdy te dwie wartości są równe
- + `False`, gdy te dwie wartości nie są równe

Operator `==` czytamy „równa się”

Operatory porównania

```
tekst1 = input('Wpisz cokolwiek: ')\ntekst2 = input('Wpisz jeszcze coś: ')\nif tekst1 == tekst2:\n    print('Wpisano dwa razy to samo:')\n    print('tekst', tekst1)
```

Wpisz cokolwiek: łoś

Wpisz jeszcze coś: łoś

Wpisano dwa razy to samo:

tekst łoś

Proszę sobie dobrze zapamiętać :-)

= przypisuje

= = porównuje

Operatory porównania

Dzięki operatorowi `!=` program też porównuje dwie wartości. Wynik operatora `!=` to:

- + `True`, gdy te dwie wartości nie są równe, czyli są różne
- + `False`, gdy te dwie wartości są równe

Operator `!=` czytamy „nie równa się”

Operatory porównania

```
tekst1 = input('Wpisz cokolwiek: ')\ntekst2 = input('Wpisz jeszcze coś: ')\nif tekst1 != tekst2:\n    print('Wpisano dwa razy co innego:')\n    print('tekst', tekst1, 'i tekst', tekst2)
```

Wpisz cokolwiek: łoś

Wpisz jeszcze coś: drugi łoś

Wpisano dwa razy co innego:

tekst łoś i tekst drugi łoś

Operator in i not in

Operator in

Dzięki operatorowi in program sprawdza, czy jeden łańcuch zawiera się w drugim łańcuchu. Wynik operatora in to:

- + **True**, gdy łańcuch po jego lewej stronie zawiera się w łańcuchu po jego prawej stronie
- + **False**, gdy łańcuch po jego lewej stronie nie zawiera się w łańcuchu po jego prawej stronie

Operator in czytamy „zawiera się w”

Operator in

```
tekst = input('Wpisz cokolwiek: ')\nif 'łóś' in tekst:\n    print('Wpisano tekst', tekst)\n    print('W tym tekście jest wyraz "łóś"')
```

Wpisz cokolwiek: miłość

Wpisano tekst miłość

W tym tekście jest wyraz "łóś"

Operator not in

Dzięki operatorowi not in program też sprawdza, czy jeden łańcuch zawiera się w drugim łańcuchu. Wynik operatora not in to:

- + **True**, gdy łańcuch po jego lewej stronie zawiera się w łańcuchu po jego prawej stronie
- + **False**, gdy łańcuch po jego lewej stronie nie zawiera się w łańcuchu po jego prawej stronie

Operator not in czytamy „nie zawiera się w”

Operator not in

```
tekst = input('Wpisz cokolwiek: ')\nif ' ' not in tekst:\n    print('Wpisano tekst', tekst)\n    print('W tym tekście nie ma spacji')
```

Wpisz cokolwiek: **ciełęcina**

Wpisano tekst **ciełęcina**

W tym tekście nie ma spacji

Operatory logiczne

Operator logiczny or

Dzięki operatorom logicznym jedna instrukcja warunkowa sprawdza wiele warunków naraz

Operator or czytamy „lub”. Wynik operatora or to:

+ **True**, gdy dowolny z jego dwóch argumentów ma wartość **True**

+ **False**, gdy oba jego argumenty mają wartość **False**

Operator logiczny or

```
litera = input('Wpisz symbol zasady: ')\nif litera == 'A' or litera == 'G':\n    print('To zasada purynowa')\nif litera == 'C' or litera == 'T' or litera == 'U':\n    print('To zasada pirymidynowa')
```

Wpisz symbol zasady: U

To zasada pirymidynowa

Operator logiczny and

Operator and czytamy „i”. Wynik operatora and to:

+ **True**, gdy oba jego argumenty mają wartość **True**

+ **False**, gdy dowolny z jego argumentów ma wartość **False**

```
hasło = input('Podaj hasło: ')
```

```
if len(hasło) >= 12 and '!' in hasło:
```

```
    print('Masz silne hasło :-)')
```

```
Podaj hasło: nie,xo338j!d32,zd
```

```
Masz silne hasło :-)
```


Operator logiczny not

Operator not czytamy „nie”

Wynik operatora not to:

+ **False**, gdy wyrażenie po jego prawej stronie ma wartość
True

+ **True**, gdy wyrażenie po jego prawej stronie ma wartość
False

Operator logiczny not

```
hasło = input('Podaj hasło: ')  
if not hasło.startswith(' ') and not hasło.endswith(' '):  
    print('Hasło nie zaczyna się od spacji ani nie kończy się spacją')
```

Podaj hasło: nie,xo338j!d32,zd

Hasło nie zaczyna się od spacji ani nie kończy się spacją

Operator logiczny not

```
tekst = 'Python to łatwy język programowania'  
if 'język' in tekst and not tekst.startswith('język'):  
    print('Tekst zawiera wyraz "język", ale się od niego nie zaczyna')  
Tekst zawiera wyraz "język", ale się od niego nie zaczyna
```

Z operatorów logicznych or, and i not można zbudować dowolne wyrażenie logiczne

Instrukcja warunkowa if, operatory porównania `==` i `!=`,
operatory in i not in, operatory logiczne or, and i not
Powtórzenie

Instrukcja warunkowa, operatory ==, !=, in, not in, or, and i not

```
hasło = input('Podaj hasło: ')
if hasło == "":
    print('Puste hasło')
if hasło != 'xyzzy':
    print('Niepoprawne hasło')
if ' ' in hasło:
    print('Hasło zawiera spację')
if ' ' not in hasło:
    print('Hasło nie zawiera spacji')
```

```
if ' ' in hasło or '\t' in hasło:
    print('Hasło zawiera biały znak')
if '*' in hasło and '#' in hasło:
    print('Bardzo silne hasło :-)')
if ('xyzzy' in hasło and
    not hasło.startswith('xyzzy')):
    print('W hasle ukryto "xyzzy"')
```

Postęp wykładu

Przedstawiłem wam:

- + Funkcję **print**
- + Funkcje
- + Łańcuchy znaków, czyli teksty
- + Instrukcję przypisania
- + Słowa kluczowe
- + Funkcję **input**
- + Indeksowanie łańcuchów
- + Wycinki łańcuchów

- + Dodawanie łańcuchów
- + Metody łańcuchów
- + Instrukcję warunkową **if**
- + Operatory porównania
- + Operatory **in** i **not in**
- + I operatory logiczne

Teraz omówię:

- + Pliki tekstowe

Proszę mi zadać pytanie :-)

Wróćmy do zadania
na rozgrzewkę

Gdybym dostał
zadanie „proszę
narysować plik”,
narysowałbym plik
kartek

A wy?



Jestem
programistą
od 30 lat

„Open file” i „close file”
tłumaczy się „otwórz plik”
i „zamknij plik”. Nie umiałem
sobie tego wyobrazić...



...aż do stycznia
2025 r. Zwiedzałem
wtedy Muzeum
Czartoryskich

Zobaczyłem eksponat podpisany
po angielsku „File of Stanislaus
August Poniatowski” i tak się
ucieszyłem, że aż podskoczyłem
z radości



Polski podpis
brzmiał: „Teczka
Stanisława Augusta
Poniatowskiego” :-)

Już wiecie, jak wygląda plik.
„Plik” to nietrafne tłumaczenie
wyrazu „file”. Niestety jest za
późno na to, by je zmienić :-)



Pliki tekstowe

Pliki tekstowe to takie pliki, które są podzielone na linie

Linie są oddzielone znakami końca linii. W Pythonie oznacza się to przez `\n`

```
Nowak\nWójcik\nKowalska\nKowalski\n
```

(Oprócz plików tekstowych są pliki binarne, o których nic więcej nie powiem)

Otwieranie pliku — funkcja `open`

Zanim program zacznie odczytywać plik, musi otworzyć ten plik funkcją `open`

+ pierwszy argument funkcji `open` to nazwa pliku

+ drugi argument funkcji `open` to tryb otwarcia pliku:

'r' jak „read” albo 'w' jak „write”

Otwieranie pliku — funkcja `open`

Funkcja `open` zwraca plik otwarty do odczytu albo do zapisu. Na zajęciach z tego przedmiotu będziemy otwierać pliki tylko do odczytu

(Programista może pominąć drugi argument funkcji `open`. Wtedy Python domyśli się, że programista miał na myśli `'r'`. Drugi argument funkcji `open` ma wartość domyślną `'r'`)

Odczytywanie linii z pliku tekstowego — pętla for

Tak program czyta linię po linii z pliku tekstowego otwartego do odczytu:

```
for linia in otwarty_plik:  
    print(linia)
```

Pamiętajmy, że po każdym dwukropku trzeba zrobić wcięcie

Zamykanie pliku tekstowego

Kiedyś trzeba było pisać tak:

```
plik = open('nazwa-konkretnego-pliku.txt', 'r')  
for linia in plik:  
    print(linia)  
plik.close()
```

Wtedy było łatwo zapomnieć o tym, żeby zamknąć plik

Zamykanie pliku tekstowego

Już nie trzeba pamiętać o tym, żeby zamknąć plik :-)

Zalecam pisać tak:

```
with open('nazwa-konkretnego-pliku.txt', 'r') as plik:  
    for linia in plik:  
        print(linia)
```

Przetwarzanie pliku tekstowego

Pamiętajmy, że możemy zagnieżdżać wcięcia. Jeśli linia programu zaczyna się wcięciem i kończy się dwukropkiem, to następna linia zaczyna się większym wcięciem

Oto przykład programu, który przetwarza plik tekstowy:

```
with open('imiona.txt', 'r') as plik:  
    for linia in plik:  
        if linia.startswith('Dag'):  
            print(linia)
```

Dagmara ż

Dagna ż

Dagobert m

Przedstawiłem wam:

- + Funkcję **print**
- + Funkcje
- + Łańcuchy znaków, czyli teksty
- + Słowa kluczowe
- + Instrukcję przypisania
- + Funkcję **input**
- + Indeksowanie łańcuchów

- + Wycinki łańcuchów
- + Dodawanie łańcuchów
- + Metody łańcuchów
- + Instrukcję warunkową **if**
- + Operatory porównania
- + Operatory **in** i **not in**
- + Operatory logiczne
- + I pliki tekstowe

To wszystko na dziś
Dziękuję wam za uwagę :-)

Proszę mi zadać pytanie :-)

Czy ktoś z was czegokolwiek
nie zrozumiał z tego wykładu?

Do zobaczenia
na następnym wykładzie,
na którym przykłady będą po czesku :-)

Źródła zdjęć

- + Microsoft Designer: jabłko z etykietką, gruszka z etykietką, gruszka z dwiema etykietkami, plik kartek
- + Pałac Czartoryskich, Wikimedia Commons, CC BY-SA 4.0
- + Muzeum Narodowe w Krakowie, teczka Stanisława Augusta Poniatowskiego, domena publiczna